Lab program-33:

33. Construct a C program to simulate the optimal paging technique of memory management

Code:

```c
 #include <stdio.h>

#define MAX_FRAMES 3

void printFrames(int frames[], int n)
{
    for (int i = 0; i < n; i++)
    {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }

    printf("\n");
}

int main()
{
    int frames[MAX_FRAMES];
    int pageFaults = 0;

    int referenceString[] =
        {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};

    int n = sizeof(referenceString) / sizeof(referenceString[0]);

    // Initialize frames
    for (int i = 0; i < MAX_FRAMES; i++)
        frames[i] = -1;

    printf("Reference String: ");

    for (int i = 0; i < n; i++)
        printf("%d ", referenceString[i]);

    printf("\n\nPage Replacement Order:\n");

    for (int i = 0; i < n; i++)
    {
        int page = referenceString[i];
```

```
        int pageFound = 0;

        // Check for page hit
        for (int j = 0; j < MAX_FRAMES; j++)
        {
            if (frames[j] == page)
            {
                pageFound = 1;
                break;
            }
        }

        // Page fault
        if (!pageFound)
        {
            printf("Page %d -> ", page);

            int optimalPage = -1;
            int farthestDistance = -1;

            // First check for an empty frame
            for (int j = 0; j < MAX_FRAMES; j++)
            {
                if (frames[j] == -1)
                {
                    optimalPage = j;
                    break;
                }
            }

            // If no empty frame, find page used farthest in future
            if (optimalPage == -1)
            {
                for (int j = 0; j < MAX_FRAMES; j++)
                {
                    int futureDistance = 0;
                    int found = 0;

                    for (int k = i + 1; k < n; k++)
                    {
                        if (referenceString[k] == frames[j])
                        {
                            found = 1;
                            break;
                        }

                        futureDistance++;
                    }
```

```
                    // Page never used again -> replace immediately
                    if (!found)
                    {
                        optimalPage = j;
                        break;
                    }

                    if (futureDistance > farthestDistance)
                    {
                        farthestDistance = futureDistance;
                        optimalPage = j;
                    }
                }
            }

            if (frames[optimalPage] != -1)
                printf("Replace %d with %d: ",
                       frames[optimalPage], page);
            else
                printf("Load into empty frame: ");

            frames[optimalPage] = page;

            printFrames(frames, MAX_FRAMES);
            pageFaults++;
        }
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_33.exe
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:
Page 7 -> Load into empty frame:  7  -  -
Page 0 -> Load into empty frame:  7  0  -
Page 1 -> Load into empty frame:  7  0  1
Page 2 -> Replace 7 with 2:  2  0  1
Page 3 -> Replace 1 with 3:  2  0  3
Page 4 -> Replace 0 with 4:  2  4  3
Page 0 -> Replace 4 with 0:  2  0  3

Total Page Faults: 7
```